

COMET, Cellular cOverage MEasurement kiT

Stepan Tyurin, Julien Saint Martin, Xavier Lagrange

2024-07-30

Documentation - Annexe

IMT Atlantique, SRCD
IRISA Adopnet



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Contents

1	Introduction	4
2	COMET composition	4
3	Assembling COMET	6
3.1	Assembling the Raspberry PI	6
3.2	Adding the SIM8262E-M2 5G HAT	6
3.3	Assembling the control interface	7
3.4	Configuring an SSH connection	10
3.4.1	Network connection	10
3.4.2	Direct connection	11
3.4.3	Exchanging SSH keys	12
3.5	Installing COMET software	12
3.5.1	Interactive installation	13
3.5.2	comet.sh utility script	14
4	Measurements	14
4.1	COMET measurements file	15
4.1.1	Concerning comments	16
4.2	Sending AT commands	17
4.3	Various useful AT commands	17
4.3.1	Entering PIN code	17
4.3.2	Retrieving operator name	18
4.3.3	Setting network mode	19
4.3.4	Powering down the module	20
4.4	Getting measurements with AT commands	20
4.4.1	Getting cell data	20
4.4.2	Geolocating measurements	23
4.4.3	Starting a GNSS session	24
4.4.4	Time to acquire a fix	24
4.4.5	Getting positioning information	25
4.4.6	Debugging with NMEA output	26

5	The Python program	28
5.1	Measurements program	28
5.2	Converting measurements to a CORENTIN-compatible file	29
5.2.1	NMEA output logger	30
5.3	Dependencies	30
6	Control Interface use	31
7	Known problems	33
8	Limitations	34

1 Introduction

Currently the radio measurements data provided to CORENTIN is collected using proprietary solutions like Accuver Xcal-M or Viavi Advisor 800. For Xcal-M to work it needs a laptop with the software installed, connected to a smartphone with another application that allows it to gather data. While it provides a lot of different information, most of it isn't used by Corentin. Moreover, carrying a laptop is heavy and consumes a lot of battery life. The software itself is proprietary so the ADOPNET team can't make any changes to the software should the need arise and the license is expensive. Similar issues apply to the Viavi solution as well.

The objective of COMET (Cellular cOverage MEasurement kiT) is to provide a low-cost solution that only gathers data about the mobile network coverage useful for CORENTIN. It does that while being lighter, cheaper, consuming less energy and with flexible software that can adapt to CORENTIN needs.

2 COMET composition

COMET is a kit of hardware with a piece of software designed specifically for the hardware used. The kit is made of the following components:

- Raspberry Pi 4B computer (RPI in this document) with a Raspberry Pi OS
- SIM8262E-M2 5G HAT (Hardware Attached on Top) by WaveShare (5G HAT in this document)
- SIM8262E-M2 5G Module by SIMCOM (5G module in this document)
- SIM Card with an active subscription
- Radio and GPS antennas (comes with the 5G HAT)
- A battery pack by Samsung acting as power supply
- Control interface in the form of two LEDs and one push button
- Software in form of a Python 3 program and various scripts

It is important to note that the software used is made for the listed hardware, most importantly the SIM8262E-M2 5G HAT. Replacing the 5G module by another one in the SIM82xx or SIM83xx series or even an older 5G module might still work but will likely introduce unexpected bugs. Using a different module will not work as it uses a different AT command set. Using a different Raspberry Pi computer or OS should still work as long as it is compatible with the 5G HAT used and uses a Linux distribution although it is not guaranteed.

The components work together as described in figure 1. The schema in the figure assumes that the power supply has been plugged in and COMET had the time to boot.

1. The COMET software, installed on the RPI and started at boot, waits until the user presses the push button of the control interface. Once pressed, the program will start a measurement session.
2. The program will open a serial connection with the 5G HAT through the USB port. It will send AT commands through that connection to set up the 5G module and then to take measurements.

3. The 5G HAT redirects the AT commands received from the RPI to the 5G module through the M.2 connector. The 5G module processes this command and does what it requires (for example change 5G module configuration or send measurements)
4. When powered up, the module will constantly listen for signals sent by the GNSS satellites and mobile network base stations and store the data in cache. Once an AT command is received, the 5G module will take this data and formulate a response with it in a specific format.
5. The response with the measurements data is sent back to the 5G HAT. The 5G HAT redirects the response from the 5G module to the RPI through the serial connection.
6. The COMET software on the RPI receives the response with the measurements and processes it. The processed measurements are then stored in a measurements file on the SD card of the RPI.
7. The steps from (2) to (6) are repeated every second until the user presses the push button again, which will stop the measurements session and safely close the serial connection. After that, COMET will be ready for a new measurements session, should the user press the button again. Alternatively, the user can hold the button for 2 seconds to shutdown COMET.
8. Once a measurements session is closed, the measurements file is ready to be retrieved. The user can open an SSH session between their computer and COMET. They can then retrieve the measurements files by executing `comet.sh transfer` on the RPI from this SSH session. Alternatively, the files can be retrieved manually.

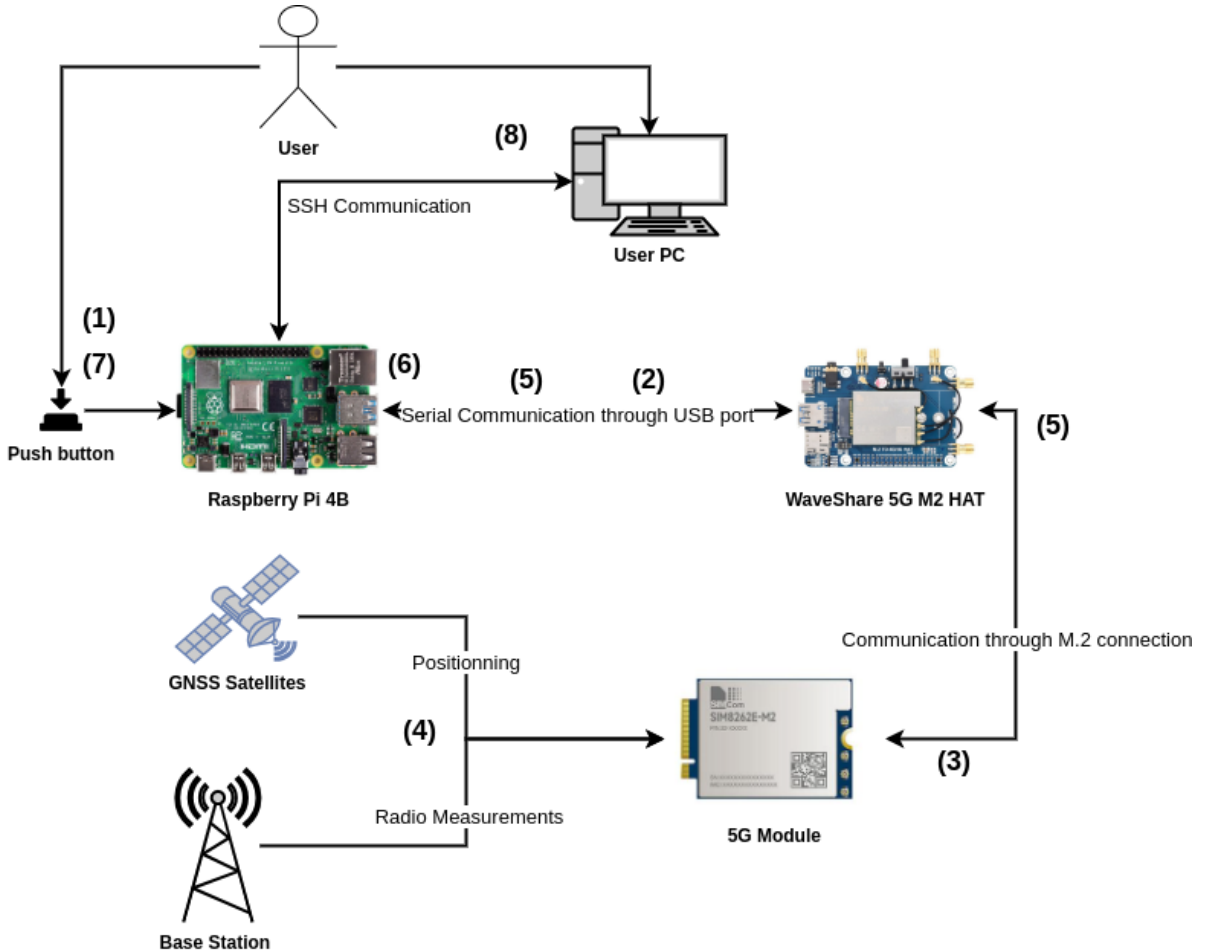


Figure 1: Interactions between COMET components during a typical measurements session

3 Assembling COMET

3.1 Assembling the Raspberry PI

First, the RPI must be assembled. An SD card carrying a Linux OS (Raspberry Pi OS is recommended) on it should be inserted in the SD card slot. A screen, mouse and a keyboard must be connected to be able to install the COMET software.

After verifying that the RPI is functioning well, four hexagonal brass standoffs must be attached to the RPI board to hold the 5G HAT. An additional RPI header must be put on the GPIO pins to extend their length.

3.2 Adding the SIM8262E-M2 5G HAT

COMET uses the SIM8262E-M2 5G HAT by WaveShare to connect the Raspberry Pi to the 5G module. Its specifications, an FAQ and some useful resources can be found on its wiki page and some more (such as the list of components coming with the HAT) on its store page.

Before installing the HAT on the RPI, a few steps must be done. First, RF cables must be connected in the slots (from ANT0 to ANT3) on the 5G module. Then the 5G module must be installed in the M2 port in the center of the board (right above the fan). After that the RF cables must be connected to the jacks on the HAT, as seen on figure 2 (doing it before attaching the HAT to the RPI is important because the RF connectors are placed on both sides of the board).

Once the cables are connected, the HAT can be attached to the RPI by gently putting it on the GPIO pins through the circular slots. This step might require some pressure on the board for the pins to go all the way through the slots. Once the pins are sticking out to their maximum length the HAT can be screwed down to the standoffs until it sits firmly on the RPI.

Next the SIM card can be inserted in its slot (it should make a very quiet click and not move unless some pressure is applied which pulls the card out) and antennas can be attached. During this step, it is important to make sure that the GPS antenna is connected to the jack that leads to **ANT3** on the 5G module, otherwise it won't work.

Lastly, both parts must be connected via USB-A 3.0 ports (any USB-A to USB-A cable or connector should work) and a power supply (like a battery giving at least 5V 3A) must be provided to the USB-C port to both RPI and HAT. The power switch must also be switched to External Power (EXT PWR). It should be possible to power the HAT through the USB-A port from the RPI, but it is untested and could prove unstable or provide insufficient power.

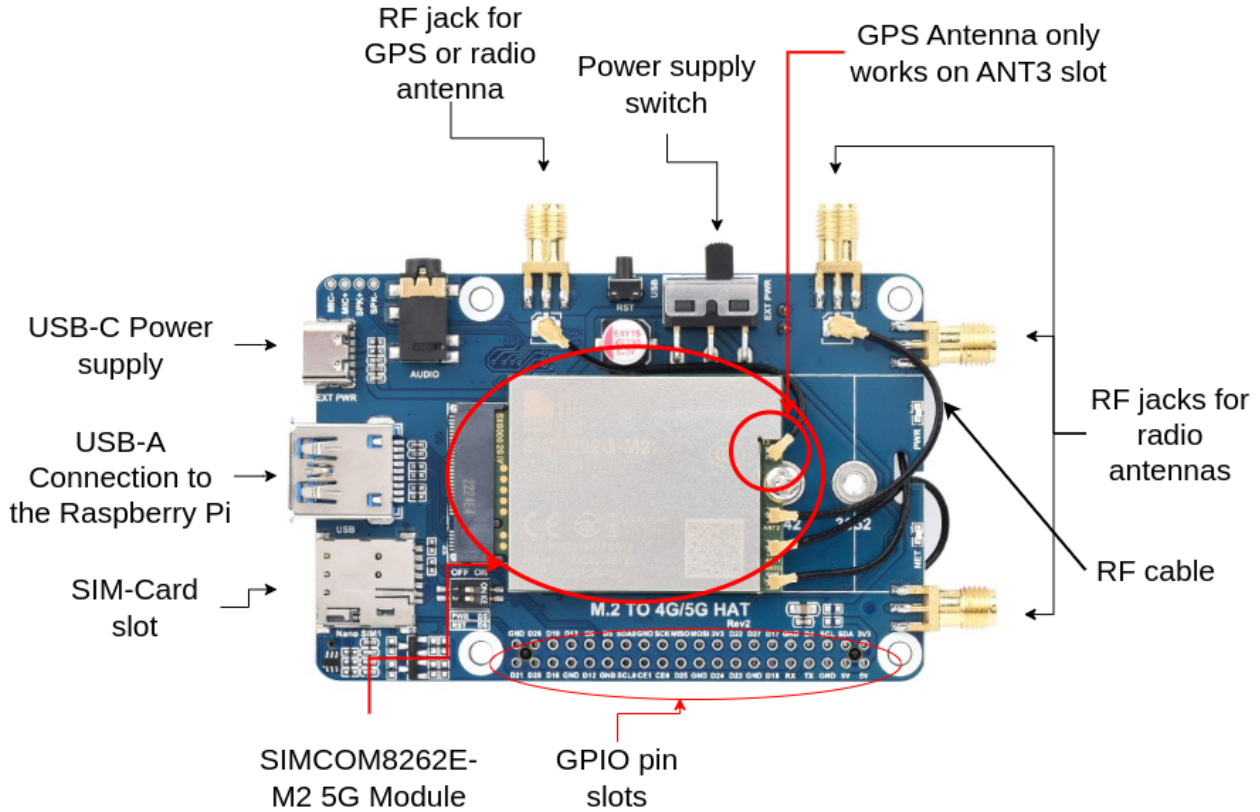


Figure 2: WAVESHARE 5G HAT

3.3 Assembling the control interface

The control interface of COMET is made up of 3 components: one red LED, one green LED and one push button. Their functionality is described in the section 6. The main measurement program is designed around using the control interface and won't work without it. For testing purposes, a command-line controlled interactive version of the program is provided, that doesn't require the control interface. This version, however, requires a constant SSH connection to another computer for control.

The LEDs and the button are controlled using the digital GPIO pins. The electric circuit can be seen in the figure 3. This is how they must be connected:

- One leg of the button must be connected to the GPIO17 (marked D17 on the HAT) pin and the other must be connected to a ground pin (all ground pins are marked GND on the HAT).
- The anode (positive side, long leg) of the red LED must be connected to the GPIO26 (marked D26 on the HAT) pin through a resistor (between 100Ω and 150Ω). The cathode (negative side, short leg) must be connected directly to a ground pin.
- The green LED must be connected to the GPIO16 (marked D16 on the HAT) pin in the same way as the red LED, but the resistor is better to be between 50Ω and 100Ω to compensate for green LED inefficiency.

The resistance on the LEDs decides how bright they will be (the less the brighter). This is important since COMET is supposed to be operated in bright sunlight and even in movement. While 150Ω or even higher can be still bright enough for the red LED, a green led is less efficient and generally

less distinguishable in sunlight. Experimentally, even with only 100Ω of resistance the green LED can become practically invisible in bright daylight.

The GPIO pins used are also defined on the software side. This means that the GPIO pins must be used as indicated in the schema (button connected to GPIO17, red LED to GPIO26 and green LED to GPIO16). If for some reason the pins cannot be used, it is possible to use a different GPIO pin by modifying the source code of the `automatic.py` module. To do that, near the bottom of the file, modify the following lines with the GPIO pins that you use:

```
with (
    Button(" GPIO17", pull_up=True, bounce_time=0.1, hold_time=2) as
        start_button ,
    LED(" GPIO16") as green_led ,
    LED(" GPIO26") as red_led ,
    CPUTemperature(min_temp=30, max_temp=100) as cpu
):
```

Not all GPIO pins can be used as some of them already have a different function. To see what pins are reserved, refer to figure 4. It is recommended to only use the GPIO pins with no other function such as GPIO27 or GPIO22, they are marked as a green circle in the figure 4. Note only the following pins have been tested (working is not guaranteed for other pins even if they are not marked as reserved): GPIO 17, GPIO27, GPIO22, GPIO23, GPIO24, GPIO25, GPIO16, GPIO26.

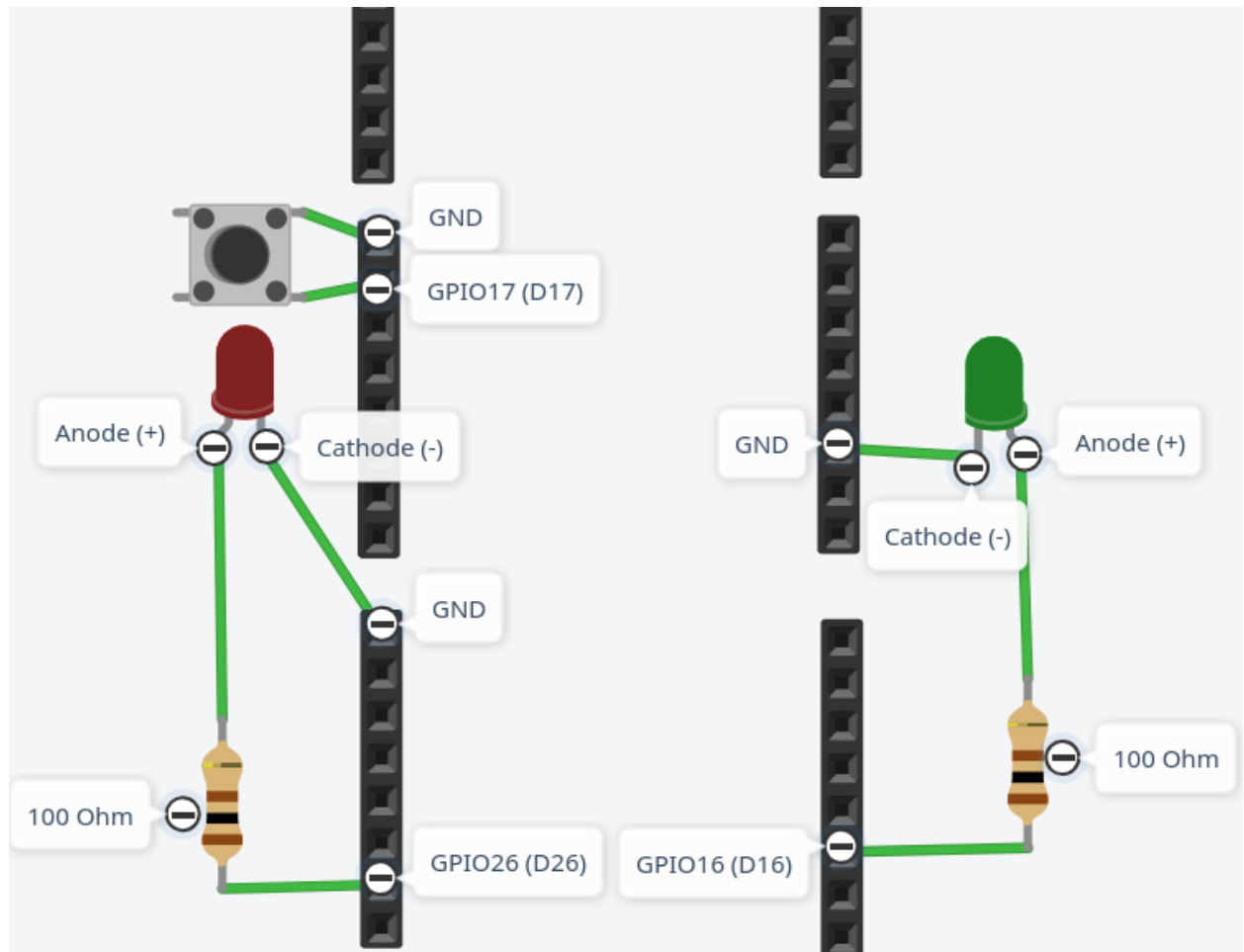


Figure 3: Schema of the control interface circuit

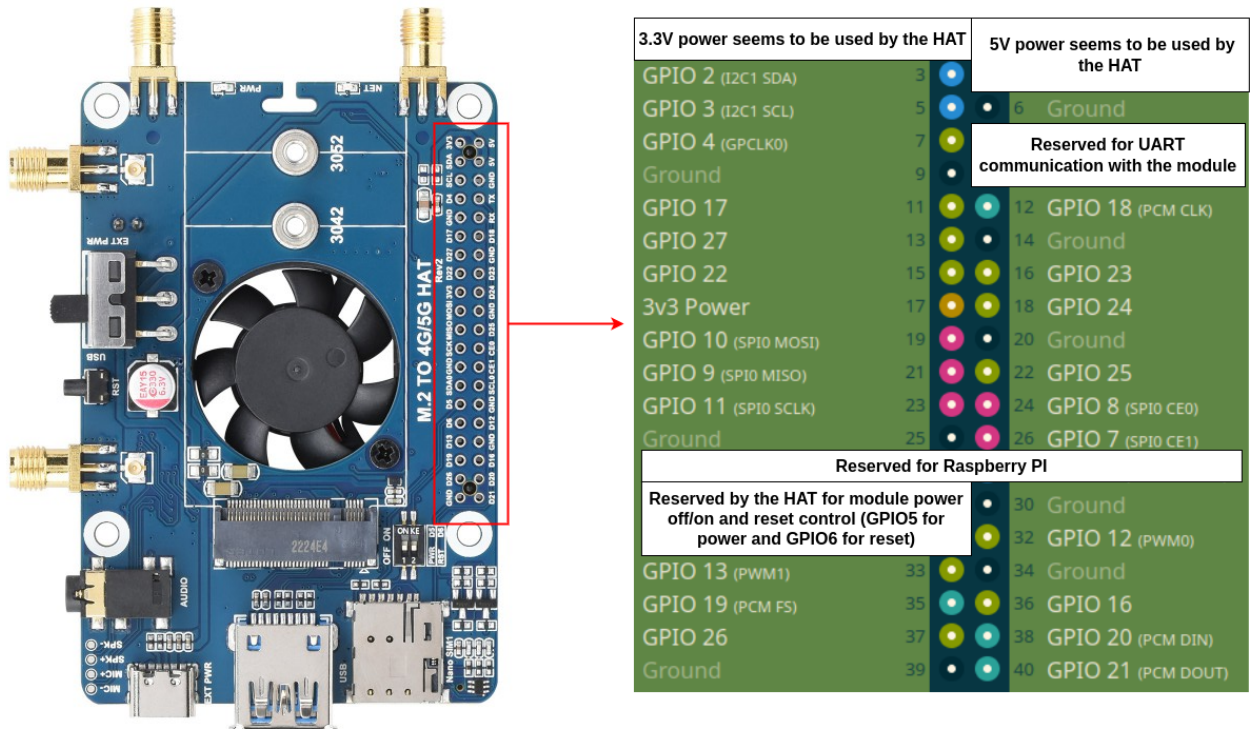


Figure 4: Free and reserved Raspberry Pi pins when using the 5G HAT

For reference, below are photos of fully assembled COMET with (figure 5) and without (figure 6) the case. In figure 6 an Ethernet cable is also connected to be able to extract the measurements done previously.



Figure 5: Fully assembled COMET in its case, ready for measurements. REPLACE PHOTO, DON'T PUT ANTENNA ON THE CASE

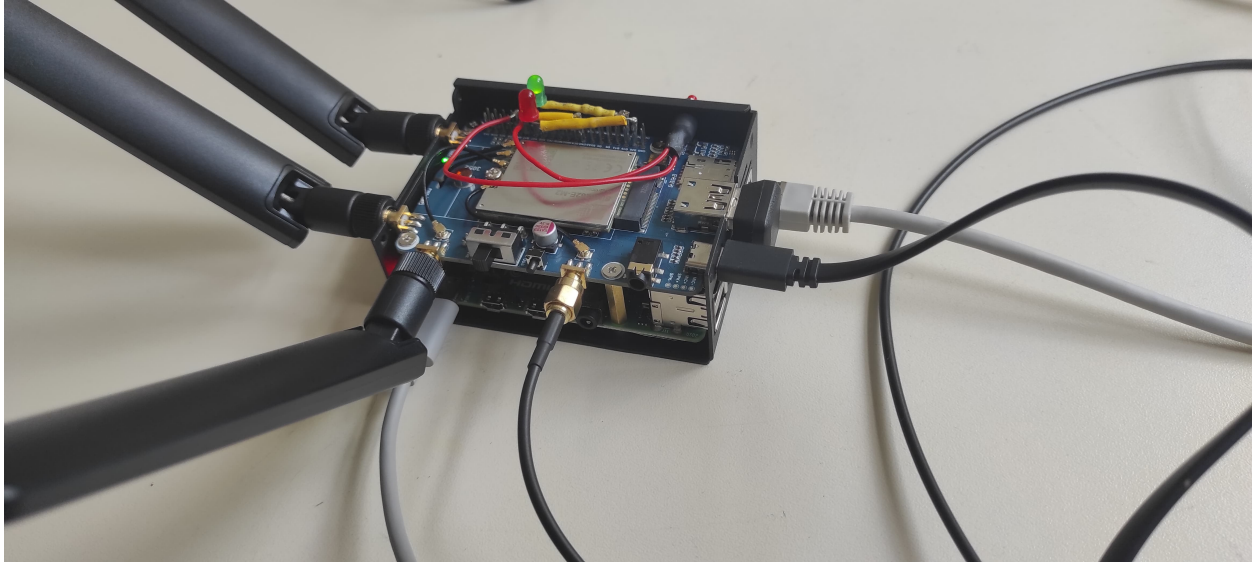


Figure 6: Fully assembled COMET without the top part of its case

3.4 Configuring an SSH connection

There are multiple ways to work with the RPI that contains COMET software and measurements files. For example having the RPI connected to a screen and a keyboard and work with all files manually, downloading files from and to internet, or reading and writing to the RPI SD card from a different computer using an adapter.

Another way, allowing for the most automation of the workflow, is configuring an SSH connection between the RPI and the user computer, which allows the use of the installation and utility scripts provided with COMET source code. In the following sections, this documentation assumes that the user has configured an SSH connection during assembly of the kit.

Essentially, an SSH connection allows for a remote access from a client computer to the server computer. This allows the client to look at files on the server, execute scripts etc. To do this, the client needs to know the IP address and the username of the server. In our case, both the user computer and the RPI can act as the client or the server depending on what is needed to be done.

Once both username and IP address known, an SSH session can be established by typing `ssh <username>@<ip>` in a terminal on the developer computer. An SSH client is installed by default on newer Windows 10 OS as well as various Linux distributions.

The SSH connection can be done in two ways: by connecting the RPI to the same network as the developer computer and using the IP addresses provided by DHCP or by connecting the two directly using an Ethernet cable and using static IP addresses configured manually. Both have their advantages and drawbacks, as explained in the sections below.

3.4.1 Network connection

When a computer is connected to local network, for example using an Ethernet cable, the DHCP server decided which IP address will be attributed to this computer, which we can't know in advance.

The RPI must first be connected to the network as well as have a screen and a keyboard attached. To learn the IP address used by the RPI, `ip addr` command can be used (when using an Ethernet

connection look for ‘eth0’ or ‘eno0’ interface). To retrieve the RPI username the `whoami` command can be used. They should be both memorized to access the RPI anytime without connecting it to a screen. An SSH server must also be enabled on the RPI either by choosing the option in the preferences tab or doing `sudo touch /boot/firmware/ssh` in the terminal, then rebooting.

The drawback of this approach is that every time a different Ethernet port is used, the IP address will change. In some circumstances it can even change when using the same port. When it happens, the procedure described in this section has to be done again and the scripts will have to be modified with the new IP address.

The advantage of this approach, however, is that this way the RPI will be connected to the Internet. This is necessary for the initial installation of the software and is allows to update the internal clock (see section 7 Known problems).

3.4.2 Direct connection

Another way to create an SSH connection is by configuring a direct connection between the RPI and the user computer and setting up static IP addresses manually. To achieve this, both must be connected with a single Ethernet cable (one end inserted in the RPI Ethernet port and the other in the developer computer Ethernet port).

Then static IP addresses on the same subnetwork must be set for the Ethernet port on both ends. How to configure a static IP address largely depends on the OS. On some Linux distributions, notably the Raspberry Pi OS, it can be done by adding a static IP address on the Ethernet interface to the `/etc/dhcpd.conf` file on the computer. The DHCP service must be restarted after modifying the file: `sudo systemctl restart dhcpd`. This must be done on both the user computer and the RPI. The lines to add to the user computer are:

```
interface eth0
static ip_address=192.168.1.1/24
```

The lines to add to the RPI are:

```
interface eth0
static ip_address=192.168.1.2/24
```

Any IP address can be used as long as it’s within the same sub network (in this case both addresses must start with 192.168.1. because it’s a /24 sub network). This configuration has to be done once on the RPI and once on every user computer that will be used for the connection. The IP addresses must stay the same for every user computer.

After doing this, these IP addresses can be used to connect to the RPI with SSH and must be added to the utility scripts as described in their respective sections in this documentation.

The advantage of this approach is that the IP addresses will always stay the same, so this needs to be done only once and the scripts can be used unmodified (if the 192.168.1.1/24 and 192.168.1.2/24 were chosen).

The drawback of this method is the additional initial configuration (which is different on each OS) but more importantly that in this configuration the RPI will not have access to the Internet, only to the user computer. This means that the software can’t be installed and the internal clock won’t be updated. There are ways to mitigate this by configuring the NAT or the ICS on Windows to enable the Internet connection sharing and mitigate this issue, but it won’t be described in this documentation as it is too OS specific.

3.4.3 Exchanging SSH keys

Some of the COMET utility scripts require passwordless SSH connection to function (mostly for copying files from and to the RPI). To achieve this, SSH keys can be exchanged between the developer computer and the RPI. Keys must be generated on the developer computer with the `ssh-keygen` command, then the public key should be copied to the RPI with `ssh-copy-id -i ~/.ssh/id_rsa username@ip` command. After that, the developer computer can connect to the RPI without a password.

This step must be done once for every user computer used and requires the IP address of the RPI to be known.

More detailed information can be found in the official Raspberry Pi tutorial and in this article on `ssh-keygen`.

3.5 Installing COMET software

COMET software contains the following:

- The `src` folder contains the Python modules making up the main measurement program. The modules are described in more details in section 5.
- `requirements.txt` contains the list of the necessary external Python packages.
- `comet.sh` provides multiple utility functions when working with COMET such as copying the measurements from the RPI to the user computer. It is also used internally by COMET to execute the main Python code.
- `install.sh` is a script that can be executed on the RPI to install all the necessary requirements and to set up all the necessary configuration so that the program could properly start on boot.
- `deploy.sh` is a script that can be executed on the user computer to fully install COMET software without manually opening an SSH session. It must be given the username and the IP address of the RPI, with a passwordless SSH connection configured, to work. Once this script completes, COMET should be ready to start measurements on reboot.

If the SSH connection was configured with an `ssh-key`, it is possible to install the software with the `deploy.sh` script. This script must be on the user computer, in the project directory (`src` directory with the Python files, `comet.sh`, `install.sh` and `requirements.txt` files must be present).

First of all, modify `USER_COMPUTER_IP` with the IP address of your computer in the `comet.sh` script. This allows for future use of its utility functions such as easy transfer of measurements and logs from the Raspberry Pi to your computer. The deployment script might first need to be given execution permissions (`chmod +x ./deploy.sh`). To deploy the software on the RPI, execute the script giving it the RPI username and IP as arguments: `./deploy.sh <yourRPIname> <your.rpi.ip.address>` (for example `./deploy_test.sh pi 192.168.1.150`).

This will copy all the necessary program files to the home directory of the RPI and execute the `install.sh` script in non-interactive mode. It will:

- Install Python 3.11 if a version below Python 3.9 was detected.

- Install the required python packages.
- Create a systemd service to allow the program to start at boot.
- Start the program.

Please note that a working internet connection on the RPI is required to complete the installation.

Once the script has finished its execution, COMET is ready for use, with LEDs indicating that the program started.

3.5.1 Interactive installation

If the ssh-key was not configured or if there are any problems with the deployment script, it is possible to install the software in interactive mode manually.

First, the folder containing the program files must be moved to the Raspberry Pi. It can be done by copying them from the developer computer through SSH using the `scp` command or by downloading them from the internet. The program files should be placed under the home directory but the exact placement is not important.

The program files contain an installation script `install.sh`. It must be given execution permissions with the `sudo chmod +x ./install.sh` command. It can be then executed by typing `sudo ./install.sh` in a terminal. Admin privileges (`sudo`) are required for the installation.

The installation script will do the following:

- COMET uses Python 3 interpreter. If the Python version installed on the Raspberry Pi is incompatible (older than Python 3.9) the user can choose to install Python 3.11 or abort installation. If the version is older than 3.11 but newer than 3.6, the user is still prompted to install Python 3.11, but the installation will continue if they refuse.
- The script will then proceed to install the required Python packages (installing the `pip` package manager if it's not present). This step, as well as Python installation, requires a working internet connection.
- A systemd service file will be generated and installed on the system, allowing the background execution. The service will be set to always start the measurement program at boot of the Raspberry Pi.
- The user will be prompted to either start the measurement program immediately or to finish installation.

Once the script has finished, the measurement program will start immediately if the user chose so, or on next boot. It is also possible to start it manually (should it fail for example) using `sudo systemctl start comet.service` command. In a similar way, it can be stopped with `sudo systemctl stop comet.service`. The LEDs will still be on (but they will reset on reboot) and generally speaking it's not recommended to do while a measurements session is ongoing. However this will free the serial connection, allowing the user, for example, using the interactive version of COMET.

3.5.2 comet.sh utility script

For the main COMET program the `comet.sh` script can be used without any modification. However, to profit from its utility functions, it must first be modified. The first lines of the script contain three variables giving the information about the user computer.

- `USER_COMPUTER_IP` should give the IP address of the user computer (for example `USER_COMPUTER_IP=10.51.0.147`)
- `USER_COMPUTER_USERNAME` gives the name of the user on the computer from which the SSH connection is established (`USER_COMPUTER_USERNAME=stepan-tyurin`)
- `USER_DIR` gives the path to the directory where the COMET source files are stored and where the measurements will be transferred from the RPI. If updating COMET files from the user computer is not envisioned, can be any directory.

Note that when using the `deploy.sh` script, the user name and directory in the `comet.sh` script will be modified automatically with current user and directory. The IP address still needs to be modified manually before executing the `deploy.sh` script.

This script must be executed on the RPI and has 4 main functions:

- Running the program: `./comet.sh run` to manually run the standard automatic version that uses the control panel with LEDs and button for interaction or `./comet.sh run-interactive` to run the command line version that doesn't require the control panel. See modules `automatic.py` and `interactive.py` in the section 5 for more details.
- Updating the program: `./comet.sh update` to update all program files (the files that will be used for update should be located in the directory indicated by the `USER_DIR` variable) or `./comet.sh update-full` to update Python requirements as well (requires internet connection on the RPI). Note that this will also update the `comet.sh` script while it's being executed. If it was modified on the user computer it can display some errors (but the files will be updated correctly). In some rare cases it can also start the measurement program in the terminal, in which case it should be stopped with a keyboard interrupt.
- Remove all temporary files: `./comet.sh clean` to remove all measurements files from the RPI, `./comet.sh clean-full` to remove the logs as well.
- Transfer the measurements files, converted "cev" files and the logs to user computer: `./comet.sh transfer` will transfer these files to the directory indicated by the `USER_DIR` variable and will create "cev", "measurements" and "logs" folders there if they don't exist.

4 Measurements

The main objective of COMET is to produce a file of measurements compatible with CORENTIN visualization. While CORENTIN accepts Xcal AOF files, it actually discards most of its measurements before doing the cell association processing. The information that is actually used is explained in CORENTIN documentation (part 5.4 Extraction of data from AOF main input file). To summarize, the data needed is:

- TAC, EARFCN, PCI, PLMN code of the serving cell.
- Radio measurements about the serving cell, such as RSRP, RSSI, RSRQ and CINR.

- RSRP, RSSI, RSRQ, CINR, as well as the PCI and EARFCN of all neighboring cells received.
- Positioning coordinates and timestamp for all measurements.

4.1 COMET measurements file

During a measurements session, all measurements done by COMET are saved to a CSV file on the RPI. The CSV file uses the '|' character as delimiter. Each measurement session creates (if it doesn't exist) a folder corresponding to the day of the measurement session (in `yyyy-mm-dd` format) in the `measurements` folder inside of the main project directory on the RPI. At the start of the session a measurement file is created inside that folder in the `HH-MM_measurement.csv` format. The measurements are written to file every second, so in case of sudden crash (for example from loss of power supply or module disconnection) most of the measurements will still be saved. They will, however, likely need to be edited manually, as crashes can partially corrupt the file. It can, for example, introduce garbage text at the end of the file

The measurement file is divided into 2 sections: The first section gives general information about the measurements session as well as the names of columns of each entry. It contains:

- Version of the measurements file (`VERSION|1.0`)
- Date of the start of the measurements session (`DATE|day-month-year hour:minute:second`)
- Mobile network operator (`OPERATOR|OPERATOR NAME`)
- An indicator if the GPS signal was lost for more than 30 seconds during the measurements or not (`GPS_LOST|YES` or `GPS_LOST|NO` or `GPS_LOST|`) if the program didn't finish correctly, for example in case of power failure)
- A line for comments on the file (`COMMENT|`).
- Column names of each entry type

The second section contains the measurements themselves. Each entry starts by its name and a timestamp and then its information. More concretely, these are the possible fields of a measurement:

```

1      GPS|TIMESTAMP|LATITUDE|LONGITUDE
2      MEASURE_SERVING|TIMESTAMP|NETWORK_TYPE|TAC|CELLID|MCC|MNC|PCID|
      EARFCN|RSRQ|RSRP|RSSI|SINR|IS_EN_DC
3      MEASURE_NEIGHBOUR_INTRA|TIMESTAMP|NETWORK_TYPE|PCID|EARFCN|RSRQ
      |RSRP|RSSI
4      MEASURE_NEIGHBOUR_INTER|TIMESTAMP|NETWORK_TYPE|PCID|EARFCN|RSRQ
      |RSRP|RSSI
```

- `MEASURE_SERVING` contains the information about the serving cell.
- `MEASURE_NEIGHBOUR_INTRA` and `MEASURE_NEIGHBOUR_INTER` contains the information about the neighboring cells that were also received by COMET. `MEASURE_NEIGHBOUR_INTRA` corresponds to the cells that work on the same EARFCN as the serving cell. The serving cell also appears as the first `MEASURE_NEIGHBOUR_INTRA`.
- GPS contains the positioning coordinates of the measurement

Possible values of each column can be found in the table 1.

Column name	Description	Example
TIMESTAMP	The exact (to the second) date of the measurement	2024-06-11 15:16:14
LATITUDE	Latitude of the measurement in decimal format	48.119814
LONGITUDE	Longitude of the measurement in decimal format	-1.6295829
NETWORK_TYPE	The network type can be LTE, 5G-SA or 5G-NSA (paired with LTE)	LTE
TAC	Tracking Area Code as an hexadecimal integer	0xC0FA
CELLID	Cell ID, a positive integer	159130624
MCC	Mobile Country Code, a positive integer	208
MNC	Mobile Network Code, a positive integer	10
PCID	PCI of the cell, a positive integer	75
EARFCN	EARFCN of the cell, a positive integer	2825
RSRQ	Measured RSRQ on the cell. Floating number with with 1 decimal digit of precision. Can be between -20 and -3 dB	-18.1
RSRP	Measured RSRP on the cell. Floating number with with 1 decimal digit of precision. Can be between -140 and -44 dB	-109.0
RSSI	Measured RSSI on the cell. Floating number with with 1 decimal digit of precision. Can be between -120.0 and 0.0 dBm	-72.3
SINR	Measured SINR (same as CINR) on the cell. An integer between -20 and 30 dB. The saved value is that given by the module, but a conversion might need to be done to find the real value, see a notice about SINR in the section 4.4.1 for more details.	-4
IS_EN_DC	Gives the information if this serving cell was received while in EN-DC mode (dual connectivity LTE-5G. Can be YES or NO.	NO

Table 1: COMET measurements file values description

A single measurement must have at least `MEASURE_SERVING` and `GPS` entries associated with it. A measurement can be defined as having the same timestamp or separated by the `GPS` entry (each measurement always has one and only one `GPS` entry at the beginning).

These files can be retrieved from COMET to the user computer by connecting to the

4.1.1 Concerning comments

Generally speaking, the converter will ignore any line that does not start with a reserved word (reserved words are all the first word of each line in the header). So it is possible to insert a line of comment anywhere, for example to write the `COMMENT` section of the header on multiple lines. To avoid the risk of accidentally using a reserved word, it is recommended to prefix comment lines with a comment symbol such as `#` or `//`. Inline comment are not allowed.

The reserved words are: `HEADER`, `VERSION`, `DATE`, `OPERATOR`, `GPS_LOST`, `COMMENT`, `MEASUREMENTS`, `GPS`, `MEASURE_SERVING`, `MEASURE_NEIGHBOUR_INTRA`, `MEASURE_NEIGHBOUR_INTER`

4.2 Sending AT commands

While sending AT commands using a tool like minicom is as easy as typing the command and pressing enter, a specific set of rules should be followed when sending AT commands at receiving responses in Python.

- Every line of text (a command) must end with `"\r\n"` and encoded to bytes before writing it to the serial port.
- Before sending the next command, a full response must be waited for and read. Usually the last line of the response is either "OK" or "ERROR" but not all commands follow that rule. Refer to the command set manual to check the response format.
- Some commands, like `AT+CPIN=XXXX`, might take a few seconds before taking effect. In some specific cases it is better to wait for a few seconds before sending more commands to avoid errors.
- All responses are given in the form of text separated in lines. Because of this, returned data often needs to be processed in a specific way depending on the command. Often a simple integer or float conversion is enough but in some cases a custom conversion method is required.
- In some cases, string responses (like names) can be returned surrounded by quotes or not, depending on the command.

4.3 Various useful AT commands

In order to obtain this information, COMET relies on AT commands that are sent by the Python program to the module. Detailed information about all commands that can be used in the SIM8262E-M2 5G module installed in COMET can be found the SIM82XX - SIM83XX Series AT Command Manual with a notable exception of `AT+QENG` which can only be found in Quectel AT commands manuals like the RG50xQ&RM5xxQ Series AT Commands Manual. However, values actually returned by that command are not necessarily exact to the manual, real values obtained experimentally on the SIM8262E-M2 5G module can be found below in the section 4.4.1. Generally speaking, most of the documentation for the SIM8262E-M2 5G module can be found on the WaveShare SIM8262E-M2 5G HAT wiki. The manuals cited here can also be found with COMET source code, under the `Docs` directory.

4.3.1 Entering PIN code

To be able to use most of the necessary commands, the SIM card must first be unlocked. To do this we use the `AT+CPIN=XXXX` command where XXXX is the PIN code of the SIM card.

Unlike other commands, this command takes a while to execute (multiple seconds) so it is important to wait for it to return its complete response. While the manual says that only "OK" or "ERROR" is returned, in actuality the full response in case of success looks like this when using the SIM8262E-M2 5G module:

```
OK
+CPIN: READY
SMS DONE
PB DONE
```

So unlike most other commands, it's not enough to only read the response until the "OK" line is found. It is not clear if the "PB DONE" is always the last line of the response as the manual does not elaborate on that, but that is what it seems experimentally.

When something went wrong, the response is in the format below, where `<err>` is the CME error code. What each code means can be found here.

```
ERROR
+CME ERROR: <err>
```

To double check that the SIM card was indeed activated `AT+CPIN?` test command can be sent in a loop checking if it returns `+CPIN: READY`.

4.3.2 Retrieving operator name

To better classify the data, it is useful to store the mobile operator that was used during the measurements. To get this information, `AT+COPS?` command is used. The response of this command has following format (meaning of each value can be found in table 2):

```
+COPS: <mode>[,<format>,<oper>[,<AcT>]]
OK
```

For example when COMET is successfully connected to network it can look like this:

```
+COPS: 0,0,"F SFR",7
OK
```

And when COMET is not connected (for example if the SIM card was not yet activated) it will look like this:

```
+COPS: 0
OK
```

The command can also return `ERROR` or `+CME ERROR: <err>` in case of a different problem.

Value	Description	Example
mode	The mode defining how operator is selected. 0: automatic 1: manual (requires setting operator with AT+COPS= command) 2: force deregister 3: set only <format> 4: manual/automatic (automatic if selected operator is not available)	0
format	Indicates how operator name is represented. 0: long format alphanumeric 1: short format alphanumeric 2: numeric	0
oper	Name of the operator, <format> indicates if the format is alphanumeric or numeric. In the numeric format gives the PLMN code of the operator where the 3 first digits is the MCC and the rest is MNC of the operator.	"F SFR"
AcT	Access technology selected, notable values are: 7: EUTRAN 11: NR_5GCN (NR connected to 5G core Network) 12: NGRAN (NG-RAN access technology) 13: EUTRA_NR (Dual connectivity of LTE with NR) 15: isn't documented in the manual but seems to be returned when the module is in a connected state but the signal is too weak. For more values, please refer to the AT command set manual	7

Table 2: Values return by AT+COPS? command

When saving the operator name to the measurements file, COMET will use the name given by the module directly if it was given in alphanumeric format and will try to convert it using publicly available information about operators' PLMN codes. Currently, COMET accepts PLMN codes of: Orange, SFR, Free, Bouygues Telecom. If the PLMN code is not recognized, the operator's name will default to "Unknown".

4.3.3 Setting network mode

By default, the 5G module selects the network mode (LTE, 5G-NSA etc) automatically, which isn't ideal as the network mode can change during the measurement session invalidating it. To avoid this, the AT+CNMP=<mode> command can be used, where <mode> is a number designating the network mode. Here are some of the notable modes (more modes can be found in the AT command set manual):

- 2 Automatic
- 38 LTE Only
- 71 NR5G
- 109 LTE+NR5G

Unfortunately, no mode is provided to force 5G-NSA, which is the only 5G mode available in Rennes at the moment. While it is possible to get 5G-NSA measurements in automatic mode, they are sporadic and can't be used to reliably measure 5G network coverage. Because of this, COMET is initialized to LTE Only mode by default using AT+CNMP=38 command.

Changing the network mode can make the module lose network connection for a short amount of time. To wait until network is back, AT+CPSI? command is used by checking if "NO SERVICE" string is present in the response in a loop, if it is not it means that network is available. There

many other ways to check if network is available, like with `AT+COPS?` or `AT+CREG`, but `AT+CPSI?` is used as it provides the most straightforward way of checking it and makes code easier to read.

4.3.4 Powering down the module

Just like the RPI must be powered down by using `sudo shutdown -h now` command to avoid the possibility of corrupting the system, the 5G module should be powered down by using an AT command. This can be done by sending the `AT+CPOF` command before unplugging the power supply.

Note that unless the power supply is unplugged, the module will restart a short amount of time after sending this command.

4.4 Getting measurements with AT commands

4.4.1 Getting cell data

The main focus of COMET measurements is getting information about the quality of the signal received from a cell. More specifically, we're interested in quality metrics such as RSSI, RSRQ, RSRP and CINR, as well as in the identity of the cell: its PCID and EARFCN.

According to the SIMCOM AT command set manual, there is only one command providing this information: `AT+CPSI`. However, it only provides this information for the serving cell. Measuring signal quality from the neighboring cells is essential for getting more meaningful network information. The `AT+QENG` command from the Quectel AT command manual, not present in SIMCOM manuals but nevertheless usable on the SIMCOM8262E module, allows us to get both the serving and neighbor cell information. To do that, this command should be used with an argument: `AT+QENG="servingcell"` and `AT+QENG="neighbourcell"` to get data from the serving cell and from neighbor cells respectively.

With the `AT+QENG="servingcell"` command, the returned information depends on the network mode of the UE (5G-SA, EN-DC, LTE) and is returned in one line for 5G-SA and LTE modes and two lines in EN-DC mode (one line for 5G-NSA information and one for LTE information).

It is important to note that the `AT+QENG="neighbourcell"` only works in LTE mode (according to the manual) and returns a list of values for each cell it receives (including the serving cell). When in LTE mode, it also gives information about the received signal in WCDMA mode which can be discarded.

`AT+QENG="neighbourcell"` response lines can be of three different types: "neighbourcell" type corresponds to a cell in WCDMA mode and can be discarded. "neighbourcell intra" corresponds to a neighboring cell on the same EARFCN as the serving cell. There is always at least one line of this type in the response, which corresponds to the serving cell (the first "neighbourcell intra" is always the serving cell). "neighbourcell inter" corresponds to neighboring cells on an EARFCN different from the serving cell.

While command arguments and the list of returned information are true to the Quectel manual, in some cases the actual values returned on the SIMCOM module do not correspond to its description in the manual and seem to be given in the same format as described in the `AT+CPSI` in the SIMCOM AT commands manual. The actual values of the return information of the `AT+QENG="servingcell"` command in LTE mode can be found in table 3.

`AT+QENG="neighbourcell"` returns less information, lacking for example the cell TAC, MNC and MCC and as such shouldn't be used without getting serving cell's information as well if full

context is needed. The returned values follow the same format for all modes (for example RSSI has the same format no matter the mode or command argument), for values that are not in common with `AT+QENG="servingcell"` in LTE mode, please refer to the Quectel command manual.

The response of `AT+QENG="servingcell"` command in LTE mode is given in the following format:

```
+QENG: "servingcell",<state>,"LTE",<is_tdd>,<MCC>,<MNC>,<cellID>,<PCID>,<earfcn>,<freq_band_ind>,<UL_bandwidth>,<DL_bandwidth>,<TAC>,<RSRP>,<RSRQ>,<RSSI>,<SINR>,<CQI>,<tx_power>,<srxlev>
OK
```

Value	Description	Raw example	Converted value
<state>	Connection state. For detailed info refer to the manual	"CONNECT"	CONNECT
<is_tdd>	The LTE network mode, can be either "TDD" or "FDD"	"TDD"	TDD
<MCC>	Mobile Country Code (first part of the PLMN code).	208	208
<MNC>	Mobile Network Code (second part of the PLMN code). Is an integer between 0 and 65535.	10	10
<cellID>	Cell ID. An integer of 28-bit (UMTS, LTE) or 36-bit (5G NR) range	158800132	158800132
<PCID>	Physical cell ID.	82	82
<earfcn>	Returned value actually is the frequency band in string format	EUTRAN-BAND3	EUTRAN-BAND3
<freq_band_ind>	Return value is actually the earfcn, an integer	1501	1501
<UL_bandwidth>	Uplink bandwidth, an integer between 0 and 5 indicating the bandwidth from 1.4 MHz to 20 MHz. Exact values can be found in the manual	5	20 MHz
<DL_bandwidth>	Downlink bandwidth, an integer between 0 and 5 indicating the bandwidth from 1.4 MHz to 20 MHz. Exact values can be found in the manual	5	20 MHz
<TAC>	Tracking Area Code. Is given in hexadecimal format. For CORENTIN it needs to be converted to an decimal integer	0xC0FA	49402
<RSRP>	Reference Signal Received Power. Range: -140 to -44 dBm. The value is given as an integer, multiplied by 10 from the actual value.	-1102	-110.2
<RSRQ>	Reference Signal Received Quality. Range: -20 to -3 dB. The value is given as an integer, multiplied by 10 from the actual value.	-200	-20.0
<RSSI>	Received Signal Strength Indication. Range: -120.0 to 0 dBm. The value is given as an integer, multiplied by 10 from the actual value.	-659	-65.9

<SINR>	Signal-to-Interface plus Noise Ratio. Is equivalent to CINR. An integer between -20 and 30 dB.	-10	-10
<CQI>	Channel Quality Indication. Range: 1–30. 0 if not available	0	0
<tx_power>	TX power value in 1/10 dBm. It is the maximum of all UL channel TX power. The <tx_power> value is only meaningful when the device is in traffic.	-99	-999
<srxlev>	Suitable reception level for inter frequency cell.	19	19

Table 3: Values returned by AT+QENG="servingcell" command in LTE mode

An important note concerning the SINR: According to the manual, the value given by the module must be converted. The conversion formula given by the manual ($Y = (1/5) \times X \times 10 - 20$, where X is the queried value and Y the real value of SINR) gives results that can seem unrealistic (a range of -60 to 40), making most of experimentally obtained values very low. An inverse formula ($Y = ((X + 20)/10) * 5$, where X is the queried value and Y the real value of SINR) gives results with a range of 0 to 25, which work well with the correspondence to the signal quality that can be found online. However, SINR is not standardized, so it can't be said with certainty which formula to use. For this reason, COMET directly stores the queried value as any formula can be applied in post-processing if needed.

Example of the response of AT+QENG="servingcell" command:

```
+QENG:"servingcell","CONNECT","LTE","FDD",208,10,158800132,82,
EUTRAN-BAND3,1501,5,5,0xC0FA,-1102,-200,-659,-10,0,-99,19
OK
```

Example of the response of AT+QENG="neighbourcell" command:

```
+QENG:"neighbourcell intra","LTE",1501,75,-200,-1116,-690,0,18,
5,62,8,62
+QENG:"neighbourcell intra","LTE",1501,82,-200,-1090,-796,0,21,
5,62,8,62
+QENG:"neighbourcell inter","LTE",2825,236,-197,-1154,-881,0,
14,7,14,20
+QENG:"neighbourcell","WCDMA",3075,2,32,6,146,-1160,-310,-1
+QENG:"neighbourcell","WCDMA",3075,2,32,6,240,-1160,-310,-1
OK
```

4.4.2 Geolocating measurements

In order to associate measurements to a base station, CORENTIN requires them to be geolocated with latitude and longitude. For this, COMET is equipped with a GPS antenna and uses AT commands to retrieve its position for each measurement.

NOTE: if this documentation and the AT commands manual don't provide enough information,

some more details can found in this GNSS application note. If the link doesn't work, the file can be found on the 5G HAT wiki or in COMET source files under the `Docs` directory.

4.4.3 Starting a GNSS session

To start gathering positioning data, a GNSS session must first be started. Once the session starts, it tries to get a first "fix" - positioning information from satellites. This step can take a long time as it needs to first locate GNSS satellites in the sky (SIM8262E-M2 is capable of receiving signals from all four major GNSS systems like GPS or GLONASS), then download the information they send periodically. At least 4 satellites are needed to acquire a fix but in practice the number is closer to 6-7 (although it should still be possible with less). Since AT GPS commands don't give any information until a fix, one can use NMEA output to debug (see section 4.4.6).

A GNSS session can be started using the `AT+CGPS=x,y` command where x is 1 to start the session or 0 to end it. The y argument refers to the mode of the GNSS session and is comprised between 1 and 3 where:

- 1 – Standalone mode
- 2 – UE-based mode
- 3 – UE-assisted mode

In theory UE-based and UE-assisted modes should be faster in acquiring a fix as they partially rely on the network to make it easier. However they require a completely different approach in extracting the positioning information and are hard to integrate in the current framework of COMET. For that reason COMET always uses the standalone mode by starting GNSS session with `AT+CGPS=1,1` command.

To be more technical, the standalone mode is the only one making the positioning information available with the `AT+CGPSINFO` command: UE-based mode automatically sends the position to the AT commands port every second, making it hard to capture while other commands are also being sent and responding, while UE-assisted mode only sends positioning information the NMEA output which would require opening a second serial connection and parsing a complicated output format, updated every second.

4.4.4 Time to acquire a fix

The time it takes to get the first fix and the ability to get it at all depends on many factors. In acquiring a fix, by far the most impactful factor is physical obstruction of the GNSS satellites. The GPS antenna must be placed outside, away from buildings, and pointing to the sky (black side up when using the patch antenna supplied with the 5G HAT). Proximity to walls and a ceiling or any other non-transparent object above the antenna can render the fix impossible. The antenna **must** have a large enough part of the sky directly visible to detect enough satellites. This is why high buildings in proximity are often the reason why the fix can't be acquired: the module can't see enough satellites as some of them are blocked by buildings.

It also important to put the antenna away from the kit itself: it creates interferences which have a very significant effect on the signal reception. Place the antenna at least 30-50 centimeters away from the module.

Another factor can be bad weather, although it's still possible to get a fix even when it's raining. It is also important to make sure that the antenna stays stable and is always pointing up.

To summarize, the antenna needs to have direct vision of the satellites. It's important to make sure that it can see as much of the sky as possible without too many obstructions.

Depending on the antenna type, putting it on a metallic surface can make signal slightly stronger. This is recommended when using a patch antenna, such as the one supplied with the 5G HAT, as it helps signal reception. Beware not to use the top or sides of the metallic case of COMET. The kit will cause interferences and the negative effects of proximity will significantly outweigh the advantages of using a metallic surface. When a fix is already acquired the signal might still stay good enough to get coordinates but acquiring the fix itself might prove impossible and the risk of loosing it is significantly greater.

Generally speaking, the time to first fix can be anywhere from 30 seconds to 15 minutes (and up to 30 minutes in some extreme cases). The first ever fix of the module is the longest as the module has no knowledge about the GNSS satellites or its approximate position. Subsequent fixes usually take between 30 to 200 seconds. When a fix was lost but the module is still running, reacquiring a fix should be faster, if the reason why the fix was lost was fixed (obstruction removed, antenna put back in a good position).

Overall, it should be noted that the GPS signal is one of the least stable components of COMET. While in most cases the first fix is relatively fast and the positioning will stay for the entirety of the measurement session, sometimes the fix can take very long time. If the environmental conditions are good (clear view of the sky), it is advised to be patient and wait for at least 10 minutes. If the GPS still doesn't work, it is recommended to check the COMET and especially NMEA logs, as explained in the section 4.4.6.

If the GPS regularly takes long time (more than 5 minutes) to be acquired or seemingly randomly doesn't get acquired at all, it can indicate a problem with either the antenna or its connection to the module. If other reasons seem unlikely (such as bad environmental conditions or bad configuration), check the antenna on a different device. If the antenna works fine, the issue can come from the RF clipper cable that links the antenna jacks on the HAT to the 5G module. Try to replace it (if no additional cables are available, a cable that links the radio antennas can be used, two antennas is usually enough to get the necessary measurements). Beware not to plug and unplug these cables too often as they are rather fragile.

4.4.5 Getting positioning information

Once a fix is acquired, we can retrieve positioning information by sending `AT+CGPSINFO` command to get latitude and longitude of the current position of COMET (note that this is only possible in standalone mode). The response is given in the following format (with values explained in table 4:

```
+CGPSINFO:[<lat>],[<N/S>],[<log>],[<E/W>],[<date>],[<UTCtime>],  
[<alt>],[<speed>],[<course>]  
OK
```

Below is an example of the `CGPSINFO` response:

```
+CGPSINFO: 4807.190267,N,137.755623,W,010624,132225.0,113.0,0.0,0.0  
OK
```

When no fix is available, the command returns `+CGPSINFO: , , , , , , ,` which is used internally to wait until the first fix at start-up.

Value	Description	Raw example	Converted value
[<lat>]	Latitude of current position in DMS format. Output format is ddmm.mmmmmmm Note that seconds are given in form of a fraction of minute.	4807.190267	48°07.190267'N Or 48°07'12"N Or 48.120000 in decimal format
[<N/S>]	N/S Indicator, N=north or S=south. Is used to convert latitude to decimal format	N	N
[<log>]	Longitude of current position. Output format is dddmm.mmmmmmm. Note that if the degrees part doesn't use all 3 digits, unused digits are ignored	137.755623	1°37.755623'W Or 1°37'45"W Or -1.629167 in decimal format
[<E/W>]	E/W Indicator, E=east or W=west. Is used to convert longitude to decimal format	W	W
[<date>]	Date. Output format is ddmmyy	010624	01/06/2024
[<UTCtime>]	UTC Time. Output format is hhmmss.s	132225.0	13:22:25
[<alt>]	MSL Altitude. Unit is meters.	113.0	113.0
[<speed>]	Speed Over Ground. Unit is knots.	0.0	0.0
[<course>]	Course in degrees.	0.0	0.0

Table 4: Values returned by AT+CGPSINFO command

4.4.6 Debugging with NMEA output

When the module doesn't have a GNSS fix, most GNSS-related AT commands execute without error, simply returning empty values. A lot of factors are at play in acquiring a fix and it can take up to 30 minutes for a first fix. Because of this, it can be hard to understand the reasons why the module doesn't acquire the fix.

To see more detailed GNSS information before a fix, it is possible to inspect the NMEA data. Most importantly, it shows the number and IDs of the GNSS satellites detected (a fix is typically acquired after some time when 5, 6 or more satellites are detected). A lack of NMEA data can also indicate problems in GNSS configuration of the module. Generally speaking, as long as the amount of satellites grows over time (even if at the pace of 1 satellite every 5 minutes), the fix will be acquired eventually, even if taking longer than expected. The time it takes to find the satellites, however, can indicate some issues such as bad environmental conditions or bad assembly.

The NMEA output needs to be configured once with AT commands before being able to read it. It can be done either using a tool like minicom connecting to the serial port (`minicom -D /dev/ttyUSB2`) or using the interactive version of COMET (by running `python interactive.py` or `./comet.sh run-interactive`).

First, the GNSS session needs to be stopped by sending `AT+CGPS=0` command. Then, the NMEA output format needs to be configured using `AT+CGPSNMEA=<nmea>` where `<nmea>` is an integer number between 0 and 262143 converted from a 17 bit binary number where each bit activates or deactivates certain output data. What each bit decides can be found in the AT command set manual (in the section dedicated to the command). An example of sufficient configuration is

`AT+CGPSNMEA=134655` which allows us to see relevant satellite information. The output rate can then be set to 1Hz by sending `AT+CGPSNMEARATE=0` (setting it to 1 makes it 10 Hz which is harder to read by a human).

After these manipulations, the module needs to be **rebooted**. After reboot, the GNSS session must be started again, by sending `AT+CGPS=1,1` command or by simply starting a measurements session.

Once the configurations are complete, the module will send the NMEA output to a serial port (with 5G HAT used in COMET it's `/dev/ttyUSB1`). When the COMET software is installed using `deploy.sh` or `install.sh` scripts, COMET will by default use this port to log the raw NMEA output to the `raw_nmea.log` file and the amount and type of satellites in a more human-readable format to the `nmea_logger.log` file. It does this thanks to a systemd service (installed via the script) that starts the `NMEALogger.py` Python script (see section 5.2.1) at boot to log the output. It is also possible to see the execute the script manually by doing `python NMEALogger.py` in a terminal on the RPI. This will show the raw output in the terminal as well as log the satellites to the log file. Another option is opening the serial connection manually using `minicom` or a similar tool (with 5G HAT used in COMET it can be done with `minicom -D /dev/ttyUSB1`) in a terminal. Note that if the the logger was installed via the `deploy.sh` or `install.sh` scripts, the systemd service will first have to be stopped to be able to access the NMEA serial port. This can be done by doing `sudo systemctl stop nmea_logger` in a terminal.

In all cases, the raw NMEA output gives a lot of different information every second. The most important line for debugging starts with `$xxGSV` (xx depends on type of GNSS, for GPS it's `GPGSV`), indicating the amount of detected satellites, their ID and signal quality. The output format of this line is as follows:

`$—GSV,x,u,xx,uu,vv,zzz,ss,uu,vv,zzz,ss,...,uu,vv,zzz,ss*hh`

The meaning of each field can be found in table 5 below. When no satellites are in view, the line can look like this: `$GPGSV,1,1,0,1*54`. When 2 satellites are detected the line should look like this: `$GPGSV,1,1,02,08,31,290,14,27,66,305,38,1*6A`.

A single GSV line contains information about at most 4 satellites. If more satellites are seen, a second (or more) GSV lines will be given just below with the rest of the satellites.

Field	Name	Description	Example
x	Number of message	Total number of GSV messages to be transmitted (1-3)	1
u	Sequence number	Sequence number of current GSV message	1
xx	Satellites in view	Total number of satellites in view (00 ~ 12)	02
uu	Satellite ID	01 ~ 32 are for GPS; 33 ~ 64 are for SBAS (PRN minus 87); 65 ~ 96 are for GLONASS (64 plus slot numbers); 193 ~ 197 are for QZSS; 01 ~ 37 are for Beidou (BD PRN). GPS and Beidou satellites are differentiated by the GP and BD prefix.	08
vv	Elevation	Satellite elevation in degrees, (00 ~ 90)	31
zzz	Azimuth	Satellite azimuth angle in degrees, (000 ~ 359)	290
ss	SNR	C/No in dB (00 ~ 99), Null when not tracking	14
hh	Checksum	Checksum field is the 8-bit exclusive OR (no start or stop bits) of all characters in the sentence. Checksum consists of 2 characters and is represented as a hex number	6A

Table 5: Fields of NMEA GPGSV line

To understand other lines of the NMEA output, please refer to a NMEA reference manual like this one.

5 The Python program

The software part of COMET consists of two Python 3 programs. The main function of the first one is to measure cellular coverage by sending relevant AT commands to the 5G module and write the measurements to a file. The purpose of the second one is to convert the measurements file into a "cev.csv" file compatible with CORENTIN.

5.1 Measurements program

The Python program responsible for making measurements is made of 6 modules:

- **SerialConnection.py** provides a class of the same name that manages the proper opening and closing of the serial connection with the module and provides methods for sending an AT command and for reading its response.
- **ATCommandSender.py** provides a class of the same name that takes a serial connection as argument and provides methods for sending specific AT commands and retrieving useful measurements with them. This is important since most AT commands require personalized treatment.
- **ATResponses.py** provides classes for each AT command used for retrieving measurements. These classes store all the data returned by the command (such as latitude and longitude for AT+CGPSINFO or PCID and EARFCN for AT+QENG="servingcell"). They provide

methods for extracting this data from a line of an AT command response and for converting the data into a format that will be printed to the measurements file.

- **MeasurementsWriter.py** provides a class of the same name that manages the creation of the measurements file and provides a method for writing its header and each measurement using its corresponding class from the **ATResponses** module. It's made in a way that makes sure that measurements are saved even in case of critical failure, such as power failure.
- **automatic.py** is an executable module that manages the execution loop. It handles the control interface by controlling the LEDs and by defining the button function. When a measurement session is started by the user, it opens a serial connection with the module and sets it up by unlocking the pin and waiting until it sees a network and GPS signal. It then creates a measurements file and does measurements, saving them to the file, until the user stops the session by pressing the button again (or shutting COMET down). This module is executed automatically on boot.
- **interactive.py** is an executable module that allows the user to easily open a serial connection to the module, set it up and interact with it using AT commands in a command line terminal. When executed, it will prompt the user if they want to set up the module and how and will then enter a loop waiting for user input. The user can give an AT command which will return its response and start a measurement session of selected duration. Note that to use this script, the serial port must not be occupied by other processes so if a measurements session is ongoing it must be stopped by pressing the measurements button before starting this script
- **utils.py** module contains various utility functions that can be used in different modules. It has, for example, a function that writes a message to a logger or directly to the command terminal depending on the arguments passed. This is mostly to make the code cleaner.

For more details on each module and how they work, please refer to the source code of COMET and its internal documentation.

5.2 Converting measurements to a CORENTIN-compatible file

The specific format of the CORENTIN-compatible "cev" measurements file is not possible to produce in real time due to its dependency on the end amount of pairs PCI-EARFCN. For that reason, the main COMET program makes an intermediary measurement file in a format that can be easily converted to a "cev" file. The **CometToCevConverter.py** module allows this conversion.

This module is used at the end of a measurements session automatically to convert the measurements file immediately, but can also be used independently from COMET on any computer, for example if COMET crashed without finishing the execution (for example on power failure) and only the COMET measurements file is available. When executed manually on a user computer, it will open a "choose file" dialogue window where the user must choose a measurements file. It will then convert the file, storing it in the "cev" directory, creating intermediary folders if needed (by default the file is created in a folder with current date as name in the cev directory). Note that when this module must be executed on a computer with a desktop to see the dialogue window, so it must not be executed on the RPI from an SSH connection.

When processing the COMET measurements file, the converter module reads the measurement file once to gather "header" information like the couples (EARFCN, PCI) and then a second time to write them to a cev.csv file in the correct format. It makes sure to properly sort the pairs (PCI, EARFCN) and align the columns accordingly to the CORENTIN cev file format.

Generally speaking, this conversion doesn't apply any particular treatment to the values stored in the COMET measurements file, with the exception of CellID being converting from hexadecimal to decimal number. This is largely due to the fact the COMET stores the values with the intention of converting them to the cev format later.

The conversion is relatively fast. For around 1 hour of a typical measurements session, it takes a little under 1 second on the Raspberry Pi 4 or around 300 milliseconds on an office computer.

For more details on how the conversion works, please refer to the module's source code and its internal documentation.

5.2.1 NMEA output logger

`NMEALogger.py` is an executable Python script that logs the GNSS satellite data. It is not required for the measurements program functioning but can be useful if any troubles with the GPS occur. It listens for the NMEA output on the `tttyUSB1` serial port and extracts satellite information: amount of satellites seen, their ID, type (GPS, GLONASS, Beidou) and their SNR. This information is logged to the `nmea_logger.log` file. It also logs the entire raw NMEA output to the `raw_nmea.log` file.

When installed using `deploy.sh` or `install.sh` scripts this script will run at boot, alongside the main measurements program. This script can also be executed manually with `python NMEALogger.py`, which will show the NMEA output to the terminal in addition to logging it to files. However, since it already running in background as a systemd service, it must first be stopped to free the serial port by doing `sudo systemctl stop nmea_logger`.

`NMEALogger.py` uses Python 3.9 (due to type hinting) and requires the `SerialConnection.py` module but is otherwise independent from the rest of the software.

5.3 Dependencies

The program is tested on **Python 3.11**. It should also work on versions newer than Python 3.9 but this is not guaranteed as it has not been tested. The program will not work on versions older than Python 3.9. The measurements program is written for the Raspberry Pi OS and is only adapted for a **Linux** OS.

The program relies on a few Python packages that need to be installed for the measurements program to function:

- **PySerial-3.5** is used to open a serial connection with the 5G module, send AT commands through it and read responses.
- **Pause-0.3** is used to align each measurement to the second.
- **psutil-5.9.8** is used to log various system metrics such as memory and CPU consumption.
- **gpiozero-2.0.1** is used to manage the control interface through the GPIO pins.
- **RPi.GPIO-0.7.1** is required as back-end for gpiozero.

On the RPI, these packages must be installed directly to the system, without creating a virtual environment. Using the virtual environment can break the way gpiozero handles its back-end and the main `automatic.py` script wont be able to start.

Note that on Raspberry Pi OS, by default, Python packages must be installed using **apt** package manager which can be limiting. To be able to install packages easier, **pip3** package manager must be installed first (if not already installed) using `sudo apt-get install python3-pip`. Then, it is important to deactivate externally managed environment restriction with the following command:

```
sudo mv /usr/lib/python3.11/EXTERNALLY-MANAGED /usr/lib/python3.11/  
EXTERNALLY-MANAGED.old
```

While **apt** package manager can be used to install packages manually without doing the procedure above. However, without **pip3**, packages can't be installed all at once with correct versions using the `requirements.txt` file.

When installing COMET software with `deploy.sh` or `install.sh` scripts, all packages will be installed automatically. This will also install **pip** if it's not present (all recent Raspberry Pi OS, including the lite version, include the **pip** package manager by default).

Note that the `CometToCevConverter.py` module can work independently from the measurements program and doesn't require any external packages to work, only the native packages that come with Python 3 by default. Unlike the measurements program, it can work on Python 3.6 or newer.

It is possible to potentially bring down the Python version requirement to Python 3.6 by modifying a few lines of code in the `automatic.py` module, in the `main` section near the bottom of the script. You need to replace the lines

```
with (LED(" GPIO16") as green_led ,  
      LED(" GPIO26") as red_led ,  
      Button(" GPIO17", pull_up=True, bounce_time=0.1, hold_time=2) as  
          start_button ,  
      CPUtemperature(min_temp=30, max_temp=100) as cpu  
):
```

with a single line:

```
with LED(" GPIO16") as green_led , LED(" GPIO26") as red_led , Button(" GPIO17", pull_up=True, bounce_time=0.1, hold_time=2) as start_button , CPUtemperature(min_temp=30, max_temp=100) as cpu:
```

The multi-line "with" statement is only available in since Python 3.9 and is used in COMET for readability purposes, as it's better than doing a very long single line or multiple nested statements. Note that while it seems like the only part of the code requiring a Python version newer than 3.6, replacing this lines does not guaranties working on Python 3.6 as it has not been tested.

6 Control Interface use

The control interface of the kit is made of one push button, one green LED and one red LED. The possible actions of the button and the meaning of both LEDs are given in tables 6, 7 and 8.

The button allows the user to start and stop a measurement session as well as gracefully shutdown COMET. Note that on a very long measurements session, it might take a few seconds to actually stop the session (normally or before shutdown) as it needs to convert the measurements first.

Action	Meaning	Detailed description
Single press	Start a new measurement session or stop the current measurement session	If pressed while the module is still not initialized (red LED always on), a measurement session will start immediately once the module is ready. If a measurement session is ongoing, it will be stopped within a second and the green LED will be turned on indicating that a new session can be started again. It will also safely stop the initialization of a measurement session should it be pressed before the measurements start.
Hold for 2 seconds	Shut down COMET	Can be done at any point of execution. This will gracefully stop the execution and shut down the RPI and the module. After the button is held for 2 seconds, it will take a few moments to finish the execution (usually less than 1 second). It is important to note that it takes the RPI some time to actually fully shutdown even once the COMET LEDs are both off. The RPI has two LEDs, red (indicating power supply) and green (indicating writing to storage). It is important to wait until the green light is off before unplugging power to avoid the risk of system's corruption. This usually lasts 5 to 10 seconds. In the standard case that comes with the 5G HAT, the LEDs can be seen from the side, through ventilation holes.

Table 6: Description of possible button actions

The green LED is used to indicate readiness and normal execution of measurements.

Duration	Meaning	Detailed description
Always on	Ready to start	A new measurement session is ready to be started by pressing the push button.
Short flash every second (on less than 100ms, off for more than 900ms)	A measurement session is ongoing	The LED is on for the duration of the measurement (usually around 25-50ms but in very rare cases can go up to 300ms), then off until the next measurement. If the LED is on for too long (more than 300ms) this might indicate a problem with measurements.
On for 100ms, off for 100ms	The measurements file is being converted to a CORENTIN-compatible "cev" file	The measurements file is being converted to a CORENTIN-compatible "cev" file.

Table 7: Green LED signals

The red LED indicates when COMET is not yet ready to start measurements (at start-up, during setup of a measurements session) and lack of signal (during measurements).

Duration	Meaning	Detailed description
Always on	Initialization or no module connection	the red LED is always on during power up of the kit until the module is recognized by the RPI and has finished it's initial configuration. If this lasts for more than a minute, check if the module is properly connected to the RPI (USB connection, power supply). The red led is also on at the start of each new measurement session while the program is trying to establish a connection with the module. This can sometimes last 20 seconds or less on first boot but shouldn't be noticeable on subsequent measurements sessions.
On for 500ms, off for 3,000ms	Absence of GPS signal	This blinking can last up to 10 minutes on the start of the first measurement session while it tries to acquire the GPS signal. If this blinking is seen during measurements (while the green LED is flashing), no positioning information is found and the measurement is lost. The blinking will continue until the signal is reacquired or the measurements are stopped.
On for 3,000ms, off for 500ms	Absence of network signal	This indicates that the module is not connected to the network. Like the GPS indicator, can be seen both before and during measurements (but usually lasts for shorter amount of time).
On for 500ms, off for 500ms	Absence of both network and GPS signal	Can only be seen while measurements are being taken (flashing green LED blinking). This can happen when passing somewhere where no signal is available, like a tunnel.
On for 1,000ms, off for 1,000ms	SIM card is still locked	This can only be seen during the module set up at the start of a measurement session, when the program tries to unlock the SIM card. Usually lasts for around 3 seconds. If it's blinking for more than 30 seconds, either the PIN code given was wrong, or if the module just has been powered on, it might have been blocked (see section 7).

Table 8: Red LED signals

7 Known problems

COMET has a few problems that can lead it to behaving in a unexpected way.

Sometimes after shutting down the RPI, plugging the power supply back might not actually boot the RPI. If after a minute the LED indicators are still off, unplug the power supply and plug it back again.

After the module boots, it performs some configurations first for some time. When trying to read something from the Python serial connection during this period, it can block the connection, entering a never-ending loop of reading, with the timeout set seemingly not working. It is unclear if the problem comes from the PySerial package or from the module itself. As the module doesn't send any confirmation when this period ends, there is no fully reliable way to avoid it. In COMET, every measurement session starts with a 30 seconds waiting period before opening the serial connection. This seems to be enough for the module to finish its initialization, but it's still an arbitrary limit that could potentially be too low.

If COMET was shut down for more than a day and was not connected to network on new boot, it will use the date from last boot. The user can still use the kit as usual, but the logs and measurements will use wrong date. Because of that, the log and measurements files will be located in wrong directories and will have wrong names (using the date before last shutdown). This happens because Raspberry Pi 4 doesn't have an Real-Time Clock, meaning it doesn't track time while powered off. Connecting COMET to network will synchronize the internal clock, so it is recommended to do so before measurements.

Despite COMET being configured by default to use multiple GNSS systems (GLONASS, BEIDOU and GPS by default, GALILEO is added on setup before measurements), only GPS satellites seem to be detected and used by the module. The GPS satellites are enough to maintain a stable fix, but the module is more prone to losing it in bad conditions and takes more time to fix than if all satellites systems were used.

8 Limitations

While COMET is a capable tool given its simplicity, there are some inherent limitations due to the hardware and software it utilizes.

Using a mobile network module, COMET is limited to the output provided by the AT commands. As such, with the SIM8262E-M2 5G module, it isn't possible to make proper 5G measurements. The AT commands only give 5G measurements for the serving cell, only LTE measurements are available on neighboring cells. Unlike with a specter analyzer such as Viavi Advisor, it isn't possible to know which beam is used on a 5G connection with AT commands. In addition, it is not possible to force the module in a 5G-NSA connection, so it can only reliably work on 5G-SA networks. This makes it highly dependent on the state of 5G deployment in the area, as many operators still largely use 5G-NSA. That means that practically it is only possible to get full measurements on a 4G network.

COMET takes a relatively long time to start up. Acquiring a GPS signal can also take multiple minutes and can lose the signal when the conditions worsen (proximity of building, weather, ceilings, other objects blocking the sky).